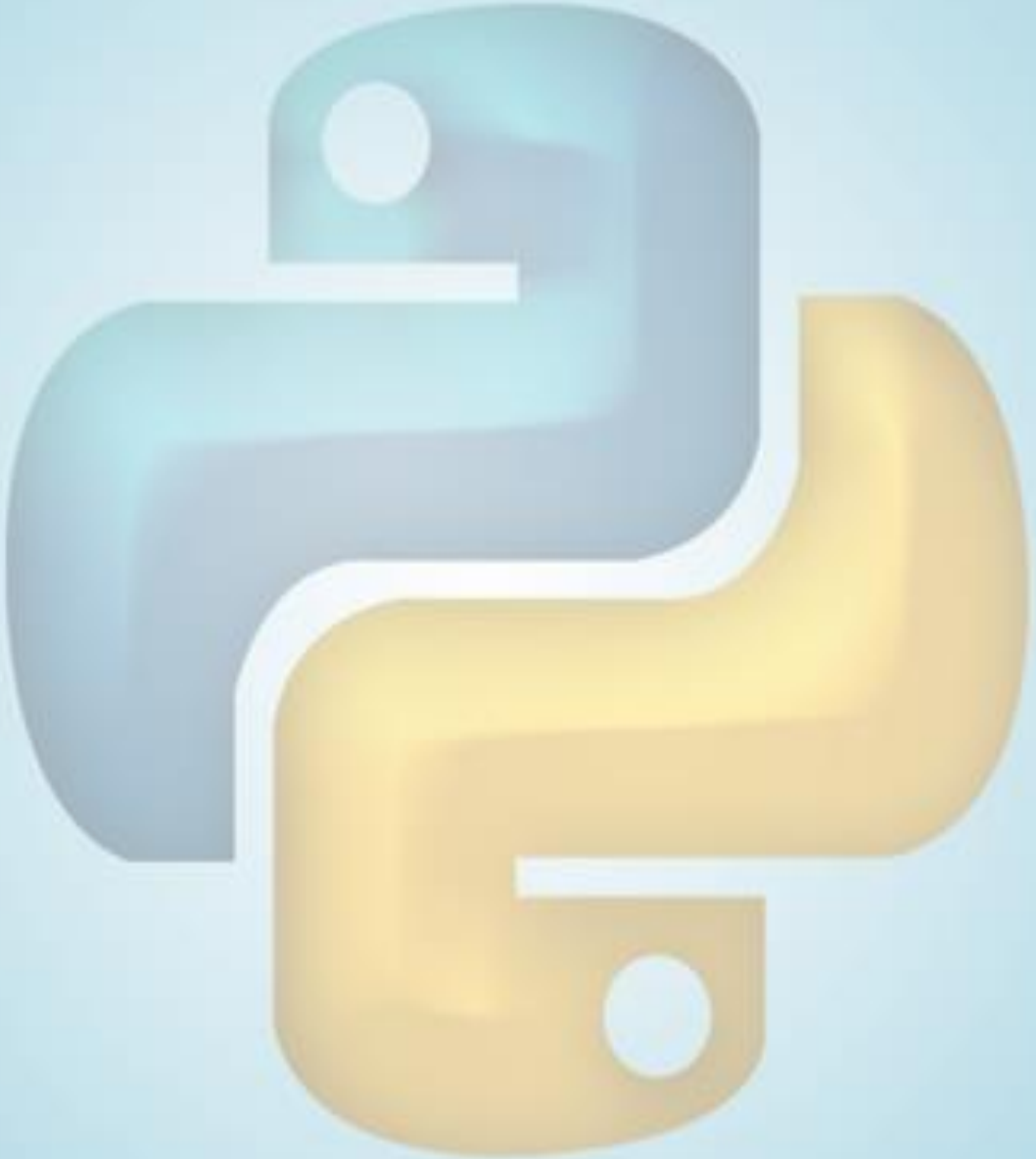
PYTHON PROGRAMMING COURSE

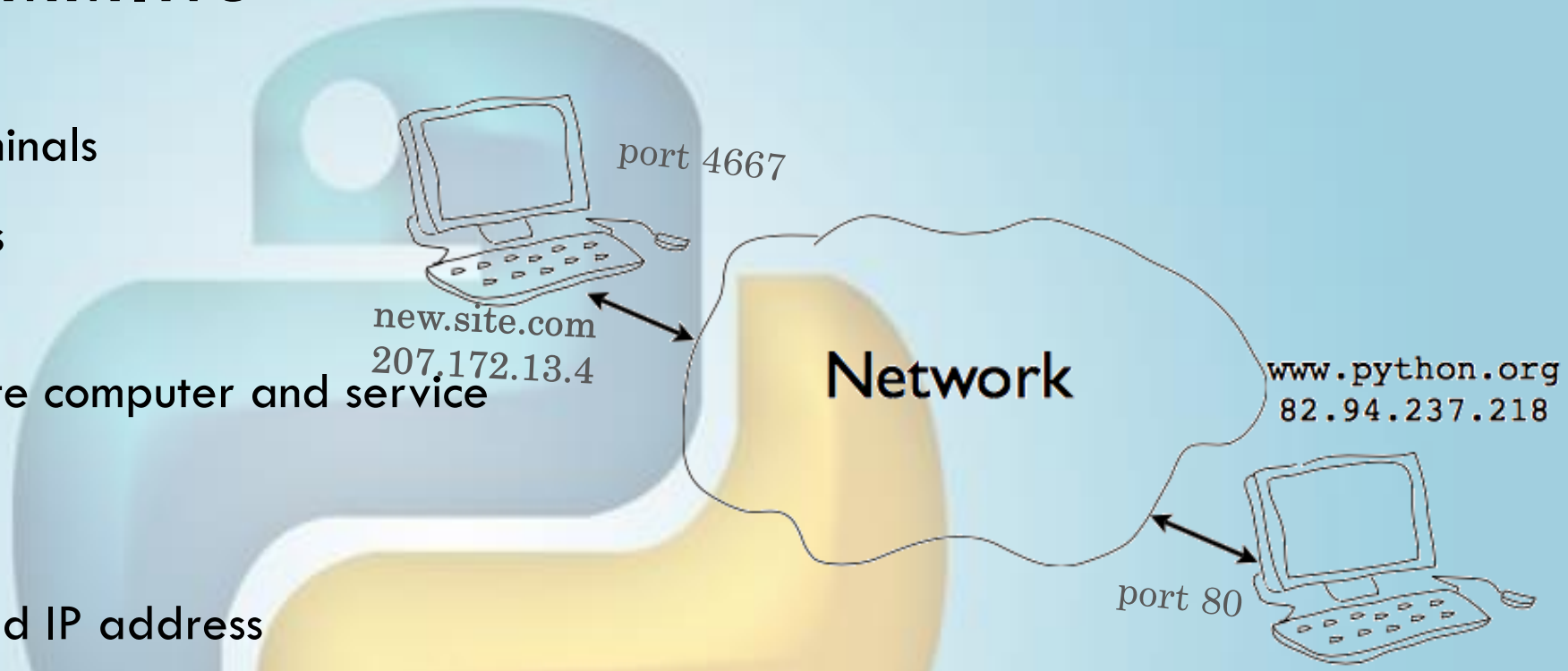
OUTLINE

- Network Programming
- Network Connection
- Client/server Concept
- Request/response
- Data transport
- Socket programming



NETWORK PROGRAMMING

- Communication between terminals
- by sending and receiving bits
- Issues
  - Addressing to specify remote computer and service
  - Data transport
- Addressing
  - Machines have hostname and IP address
  - Program/service has port numbers
  - Standard ports
  - Non-standard ports




```
21          FTP
22          SSH
23          Telnet
25          SMTP (Mail)
80          HTTP (Web)
110         POP3 (Mail)
119         NNTP (News)
443         HTTPS (web)
```

NETWORK CONNECTION

- Netstat command displays current TCP/IP network connections and protocol statistics
- Each network connection is represented by a host and port number
- In Python, it is written as tuple (host,port) => (“www.python.org”,80)
- Run netstat –a command from cmd

```
shell % netstat -a
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address
tcp        0      0 *:imaps                 *:*
tcp        0      0 *:pop3s                 *:*
tcp        0      0 localhost:mysql         *:*
tcp        0      0 *:pop3                  *:*
tcp        0      0 *:imap2                 *:*
tcp        0      0 *:8880                  *:*
tcp        0      0 *:www                   *:*
tcp        0      0 192.168.119.139:domain  *:*
tcp        0      0 localhost:domain        *:*
tcp        0      0 *:ssh                   *:*
...
```

CLIENT/SERVER CONCEPT

- Each endpoint is a running program
- Server waits for incoming connections and provide a service i.e. web, email etc.
- Clients connect to servers

REQUEST/RESPONSE CYCLE

- General networking programs use request/response model based on messages
- Client sends a request message e.g. HTTP (GET /index.html HTTP/1.0)
- Server sends back response message
- Reply message format depends on application
- Telnet
  - telnet hostname portnum

```
HTTP/1.0 200 OK
Content-type: text/html
Content-length: 48823

<HTML>
...
```

type this and press return a few times →

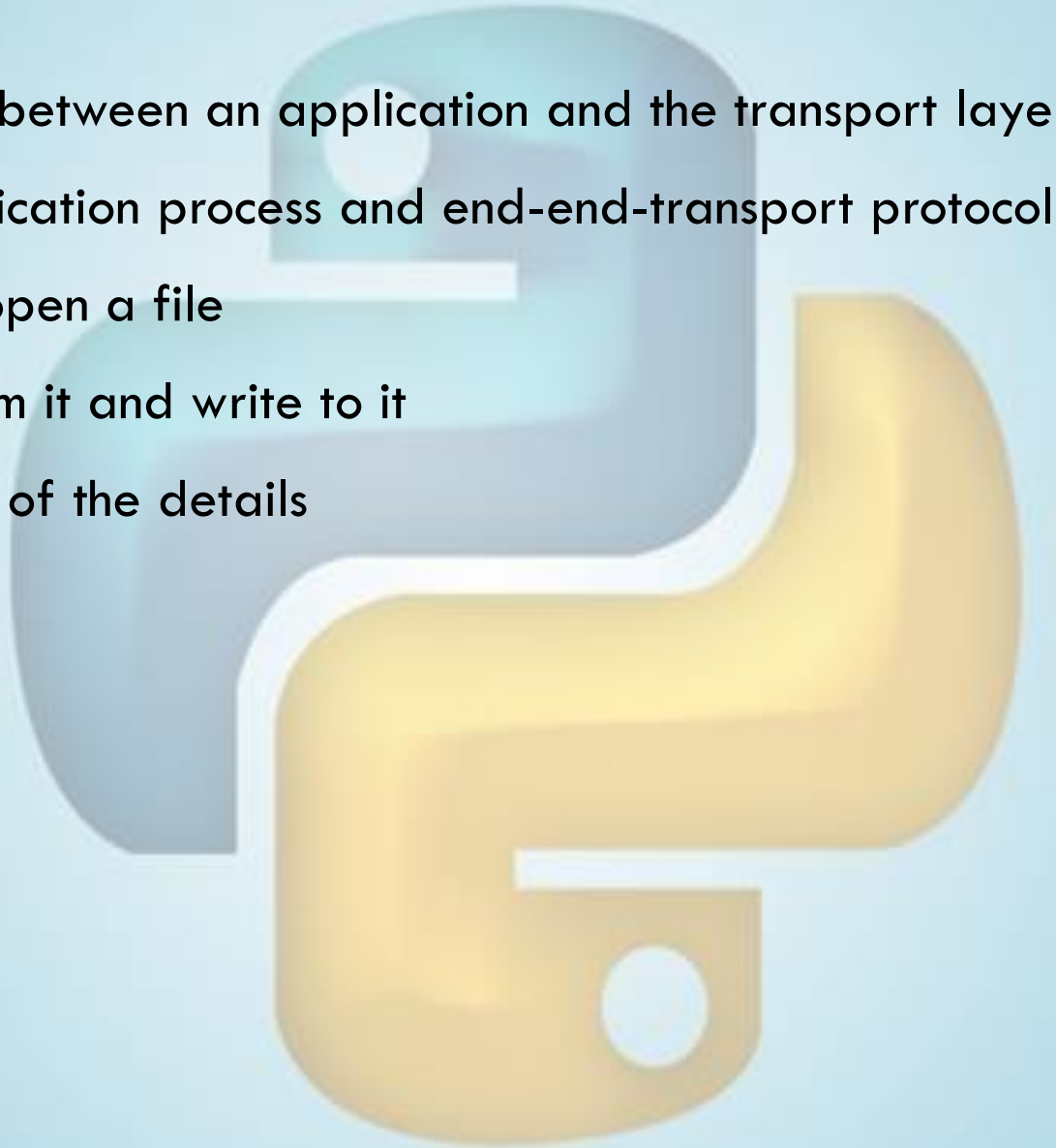
```
shell % telnet www.python.org 80
Trying 82.94.237.218...
Connected to www.python.org.
Escape character is '^]'.
GET /index.html HTTP/1.0

HTTP/1.1 200 OK
Date: Mon, 31 Mar 2008 13:34:03 GMT
Server: Apache/2.2.3 (Debian) DAV/2 SVN/1.4.2
mod_ssl/2.2.3 OpenSSL/0.9.8c
...
```

DATA TRANSPORT

- There are two basic types of communication
- TCP (Streams)
  - Computers establish a connection with each other and read/write data in a continuous stream of bytes like a file. This is the most common
  - Reliable
- UDP (Datagrams)
  - Computers send discrete packets (or messages) to each other. Each packet contains a collection of bytes, but each packet is separate and self-contained
  - Unreliable

SOCKET PROGRAMMING IS EASY

- Sockets define the interfaces between an application and the transport layer
- Socket: a door between application process and end-end-transport protocol (UCP or TCP)
- Create socket much like you open a file
- Once open, you can read from it and write to it
- Operating System hides most of the details



SOCKETS

- Socket is a communication endpoint
- Python socket module
- Allows connections to be made and data to be transmitted in either direction
- Socket basics
  - Create a socket

```
import socket
soc = socket.socket(addr_family, type)
```

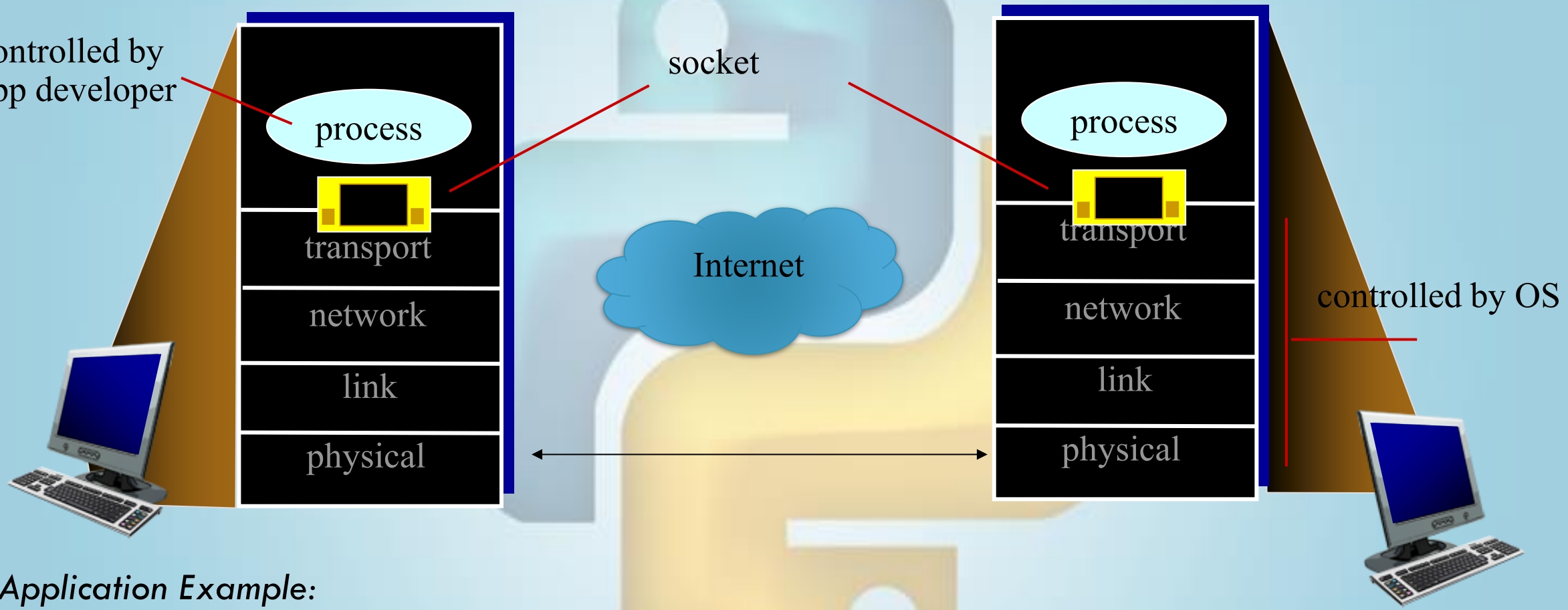
  - Address families

```
socket.AF_INET      #Internet Protocol IPv4
socket.AF_INET6     #Internet Protocol IPv6
```

  - Socket types

```
socket.SOCK_STREAM      TCP connection based
socket.SOCK_DGRAM       UDP
```

SOCKET PROGRAMMING



Application Example:

1. Client reads a line of characters (data) from its keyboard and sends the data to the server.
2. The server receives the data and converts characters to uppercase.
3. The server sends the modified data to the client.
4. The client receives the modified data and displays the line on its screen.

SOCKET PROGRAMMING *WITH UDP*

- UDP: no "connection" between client & server
- no handshaking before sending data
- sender explicitly attaches IP destination address and port # to each packet
- rcvr extracts sender IP address and port# from received packet
- UDP: transmitted data may be lost or received out-of-order
- Application viewpoint:

UDP provides *unreliable* transfer of groups of bytes ("datagrams") between client and server

CLIENT/SERVER SOCKET INTERACTION

server (running on serverIP)

create socket, port= x:
serverSocket =
socket(AF_INET,SOCK_DGRAM)

read datagram from
serverSocket

write reply to
serverSocket
specifying
client address,
port number

client

create socket:
clientSocket =
socket(AF_INET,SOCK_DGRAM)

Create datagram with server IP and port=x; send datagram via clientSocket

read datagram from
clientSocket

close
clientSocket

Example app: UDP client

include Python's socket library

create UDP socket for server

get user keyboard input

Attach server name, port to message; send into socket

read reply characters from socket into string

print out received string and close socket

Python UDPClient

```
from socket import *
serverName = 'hostname'
serverPort = 12000
clientSocket = socket(AF_INET, SOCK_DGRAM)
message = input('Input lowercase sentence:')
clientSocket.sendto(str.encode(message),
                    (serverName, serverPort))
modifiedMessage, serverAddress =
                    clientSocket.recvfrom(2048)
print (bytes.decode(modifiedMessage))
clientSocket.close()
```

Example app: UDP server

```
Python UDPServer
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET, SOCK_DGRAM)

serverSocket.bind(('', serverPort))
print ("The server is ready to receive" )
while 1:
    message, clientAddress = serverSocket.recvfrom(2048)
    modifiedMessage = message.upper()
    serverSocket.sendto(modifiedMessage, clientAddress)
```

create UDP socket

bind socket to local port number 12000

loop forever

Read from UDP socket into message, getting client's address (client IP and port)

send upper case string back to this client

Example app:TCP server

```
"""
Messenger program using TCP socket
Save as TCPServer.py
"""
import socket
def server_program():
    # get the hostname
    host = socket.gethostname() #locat host name
    port = 5000                 # port nunber above 1024
    # get instance of server socket
    server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    #The bind() function takes tuple as argument
    server_socket.bind(("", port))  # bind host address and port together

    # configure how many client the server can listen simultaneously
    server_socket.listen(2)
    conn, address = server_socket.accept()  # accept new connection
    print("Connection from: " + str(address))
    while True:
        # receive data stream. it won't accept data packet greater than 1024 bytes
        data = conn.recv(1024).decode()
        if not data:
            # if data is not received break
            break
        print("User " + str(address) + "says "+ str(data))
        data = input(' -> ')
        conn.send(data.encode())  # send data to the client

    conn.close()  # close the connection



server_program()
```

Example app:TCP client

```
"""
Messenger program using TCP socket
Save as TCPClient.py
"""

import socket

def client_program():
    host = "127.0.0.1"          #otherwise server ip address
    port = 5000                 # socket server port number

    # instantiate client socket
    client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    client_socket.connect((host, port))     # connect to the server

    #message = input(" -> ")                # input() gives error on client side, need raw_input
    try:
        input = raw_input
    except NameError:
        pass
    message = input("->")

    while message.lower().strip() != 'bye':         #stopping condition
        client_socket.send(message.encode())        # client send message to server
        data = client_socket.recv(1024).decode()    # receive response

        print('Received from server: ' + data)      # show in terminal

        message = input(" -> ")                     # again take input

    client_socket.close()                           # close the connection

client_program()
```

ENOUGH FOR TODAY